Estruturas de Dados

Hash Tables



2021/2022

Hash Tables

- As árvores binárias de pesquisa são vocacionadas para o armazenamento de informação ordenada.

- Uma *hash table* é uma estrutura de dados que pode ser usada quando a ordem da informação não é relevante
  - Inserções, procuras e remoções são feitas em tempo médio constante!



Hash Table - Introdução

- **Introdução (exemplo):**
  - Vamos assumir que estamos a trabalhar com um conjunto de números inteiros positivos pequenos (entre 0 e 65535)
  - Para guardar um conjunto destes números, podemos usar um *array a* com um número correspondente de posições inicializadas a 0.
  - As operações básicas podem ser implementadas de forma muito simples e eficaz:
    - *Inserção(i): a[i]++;*
    - *Remoção(i): a[i]--;*
    - *Pesquisa(i): a[i]!=0*



Hash Table - Introdução

- **Introdução (exemplo):**
  - Quais é que são os problemas desta solução?
    - *Se usassemos números de 32 bits era necessária uma tabela com 4 G entradas ...*
    - *Nem todos os dados são de natureza numérica (strings?)...*



Hash Table - Introdução

- Nem todos os dados são de natureza numérica??
  - Nem por isso... Toda a informação é armazenada por *bits* que podem ser interpretados numericamente.
  - P.ex:
    - Cada caracter corresponde a 7 bits (código ASCII). Por isso, uma string pode ser interpretada como um número em base 2^7=128 da seguinte forma:

"Ola" = *79* * 2^{14}+ *108* * 2^7+ *97* * 2^0 = *1308257*

↑ ↑ ↑

'O' 'l' 'a'



Hash Table - Introdução

- Um problema da abordagem anterior é que os números necessários para representar as strings são enormes:
  - Uma string de N caracteres é representada por um número com N*7 bits.
  - Para strings com 10 caracteres seriam necessários número com 70 bits!
    - Ou seja, uma tabela de dimensão $\mathbf{2^{70}}$
    - O que é muito,... mas mesmo muito, grande (1 Tera = $\mathbf{2^{40}}$).
  - É necessário contornar esta dificuldade adicional...



Hash Table - Introdução

- A ideia é encontrar uma função que transforma os dados em números de dimensão mais reduzida...
  - Qualquer função deste género é denominada por ***função de hash*** *(*ou função de escrutínio*)*.
  - Se uma função de *hash* mapear os dados em inteiros entre *0* e *X-1*, um *array* com dimensão *X* será suficiente.



Hash Table - Introdução

- Uma forma básica de mapear qualquer inteiro *N* para a gama *[0,X[* é obter o resto da divisão de *N* por *X*, ou seja
  - *H(N,X)=N mod X*
- *Uma possibilidade que surge consiste então em usar o modulo para obter, a partir do número grande, o indice desejado. Por exemplo:*

H(**“Ola”,1024) = *1308257 mod 1024=609***

Inserção(“Ola”) ⇒
a[H(“Ola”,1024)]++;



Hash Table - Introdução

- *Como é obvio, há agora um novo problema a resolver:*
  - *Existe a possibilidade de ocorrerem colisões, ou seja, strings diferentes que são mapeadas para o mesmo índice...*
  - *Ex:*

$$\text{“La”} = 76 * 2^7 + 97 * 2^0 = 9825$$

H(**“La”,1024)=609**

Logo “La” e “Ola” correspondem ao mesmo índice... E agora?



Hash Table - Introdução

- As colisões são inevitáveis, mas há várias formas de lidar com elas.
- Vamos analisar três delas:
  - Sondagem linear
  - Sondagem quadrática
  - Encadeamento



Funções de Hash

- Uma propriedade da função de *hash* é que deve ser simples (e rápida) de calcular.
- A função apresentada anteriormente não o é...
  - Os resultados intermédios podem fazer *overflow*...
  - Para uma string com 10 dígitos, precisamos de uma fórmula com o termo $\mathbf{2^{70}}$... Como é que vamos lidar com isso?



Funções de Hash

- Sabendo que:

$$79 * 2^{14} + 108 * 2^{7} + 97 * 2^{0} = ((79 * 2^{7} + 108) * 2^{7} + 97)$$

- Ou, de forma genérica:

$$A_3 X^3 + A_2 X^2 + A_1 X^1 + A_0 X^0 = (((A_3)X + A_2)X + A_1)X + A_0$$

$$a_2 = A_3 X + A_2$$

$$a_1 = a_2 X + A_1$$

$$a_0 = a_1 X + A_0$$

O cálculo pode ser feito iterativamente através de uma expressão com a forma aX +A (evita-se o uso de X^n)



Funções de Hash

- Os resultados intermédios (e final) ainda podem fazer *overflow*...
- Dado que apenas nos interessa o módulo, podemos aplicar essa operação após cada passo intermédio para reduzir a dimensão dos resultados intermédios.

$$a_n = (a_{n+1}X + A_n) \bmod M$$



Funções de Hash

```
// função de hash aceitável
public static int hash( String key, int tableSize )
{
int hashVal = 0;
for( int i = 0; i < key.length( ); i++ )
   hashVal = ( hashVal * 128 + key.charAt( i ) ) % tableSize;
return hashval;
}
```

- Uma deficiência desta função é que a operação % é demorada...
- Outro problema é que a multiplicação por 128 roda os primeiros caracteres para a esquerda (parte mais significativa do resultado), pelo que serão tipicamente cortados pela operação de módulo, não tendo qualquer impacto no resultado final.



Funções de Hash

- **Como resolver o primeiro problema (custo de tempo da operação %)?**
  - Permite-se o *overflow* natural das operações, fazendo-se um **único** *%* no final. Neste caso, deve-se ter em atenção que um *overflow* pode gerar resultado negativos.
  - O resultado não é o mesmo da função anterior, mas isso não é relevante!

```
// função de hash aceitável – 2ª versão
public static int hash( String key, int tableSize )
{
int hashVal = 0;
for( int i = 0; i < key.length( ); i++ )
   hashVal = ( hashVal * 128 + key.charAt( i ) );
return hashval % tableSize;
}
```



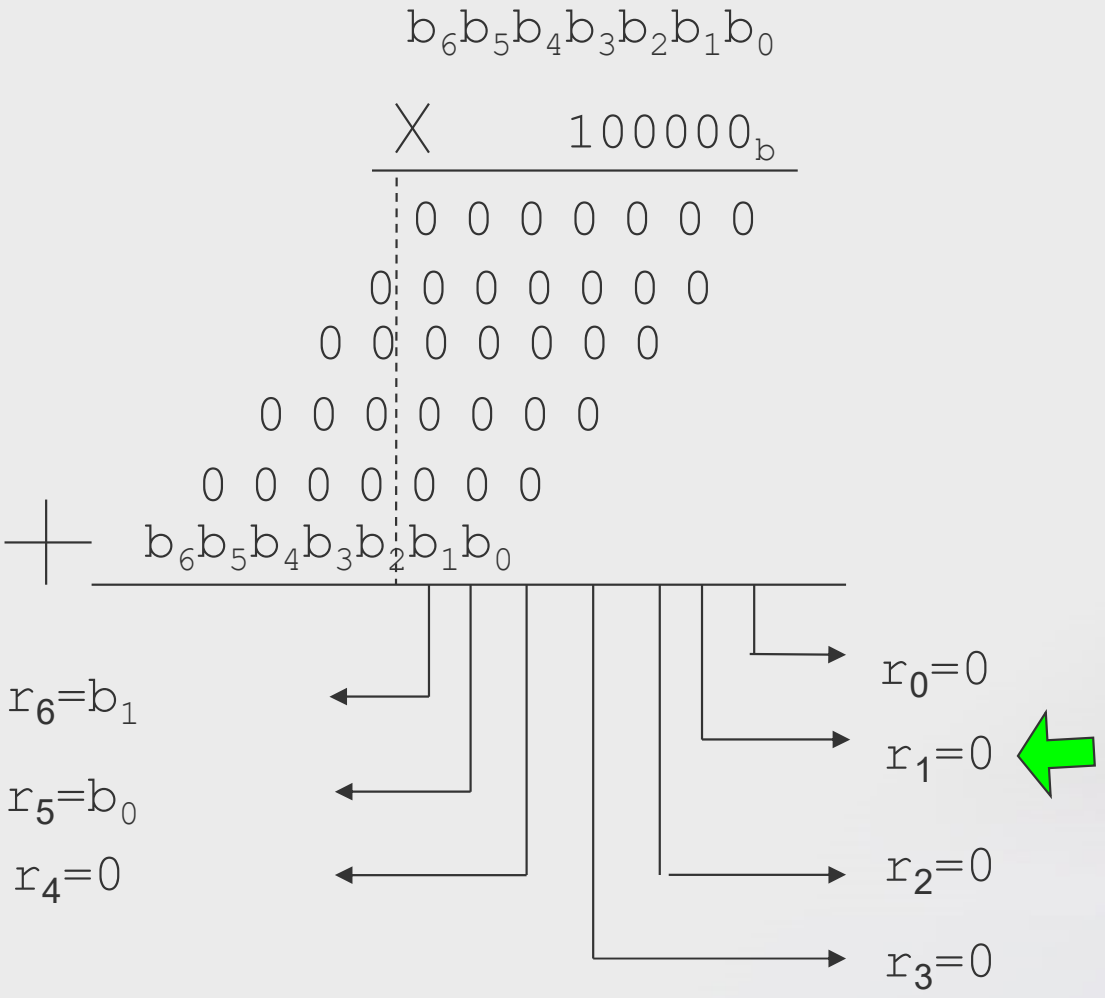
Funções de Hash

- **Como resolver o segundo problema (perda de informação em relação aos *bits* de mais significativos)?**
  - Multiplicam-se os resultados intermédios não por 128, mas sim por outro número que não efectua uma rotação perfeita. 37 é uma boa escolha.
  - Isto significa que os primeiros caracteres considerados não serão rodados tão depressa para a parte mais significativa do resultado (e dessa forma terão mais (algum) impacto no resultado final).



Funções de Hash

- Exemplo: Multiplicação por uma potência de 2: 32 (100000 b)



Os bits mais significativos são directamente dependentes de um pequeno número de bits menos significativos

Os bits menos significativos são 0 ...



Funções de Hash

- Exemplo: Multiplicação por 37 (100101b)

$b_6b_5b_4b_3b_2b_1b_0$

$\times \quad 100101_b$

$b_6b_5b_4b_3b_2b_1b_0$

$0\ 0\ 0\ 0\ 0\ 0\ 0$

$b_6b_5b_4b_3b_2b_1b_0$

$0\ 0\ 0\ 0\ 0\ 0\ 0$

$+ \quad 0\ 0\ 0\ 0\ 0\ 0\ 0$

$b_6b_5b_4b_3b_2b_1b_0$

$T(r_x)$ é dependente das mesmas variáveis que r_x

O valor de r_3 é uma função de (depende de) todos os primeiros 4 bits.

E assim sucessivamente..

$r_6=b_6+b_4+b_1+T(r_5)$

$r_5=b_5+b_3+b_0+T(r_4)$

$r_4=b_4+b_2+T(r_3)$

transporte

$r_0=b_0$

$r_1=b_1$

$r_2=b_2+b_0$

$r_3=b_3+b_1+T(r_2)$

$b_3+b_1+b_2b_0=$

$F(b_3,b_2,b_1,b_0)$



Funções de Hash

- Função de hash - Final

```
//função de hash – versão final
public static int hash( String key, int tableSize )
{
   int hashVal = 0;
   for( int i = 0; i < key.length( ); i++ )
        hashVal = 37 * hashVal + key.charAt( i );
   hashVal %= tableSize;
   if( hashVal < 0 )
        hashVal += tableSize;
   return hashVal;
}
```



Funções de Hash

- A velocidade da função de hash é importante, mas não é o único critério:

```
public static int hash(String key, int tableSize)
{
int hashVal = 0;
for( int i =0; i < key.length( ); i++ )
hashVal += key.charAt( i );
return hashVal % tableSize;
}
```

- Esta função é rápida,mas é má, porque não distribui equitativamente os indices gerados pela gama disponível.
  - *O valor da chave irá concentrar-se perto do valor médio esperado (número de caracteres* valor médio de um caracter), provocando um grande número de colisões.*
  - *Esta tendência ancentua-se com o aumento da dimensão da string.*



Colisões - Sondagem Linear

- O que fazer quando há uma colisão?
  - Mesmo que as chaves estejam bem distibuidas, as colisões são inevitáveis.
  - Dessa forma, o que fazer se a posição indicada já se encontra ocupada (presumivelmente por outra *string*)?



Colisões - Sondagem Linear

- A estratégia mais simples consiste em pesquisar sequencialmente o *array* a partir da posição original até que seja encontrada uma posição livre.
  - Esta pesquisa dá a volta do final para o inicio do *array* se necessário.



Colisões - Sondagem Linear Exemplo:

– Considere-se uma função de *hash H(N,M)=N%M*.

Insere(89);
Insere(18);
Insere(49);
Insere(58);
Insere(9);

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	



Colisões - Sondagem Linear

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

Insere(89); *H(89,10)=9*
Insere(18);
Insere(49);
Insere(58);
Insere(9);

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	



Colisões - Sondagem Linear

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

Insere(89); *H(89,10)=9*
Insere(18);
Insere(49);
Insere(58);
Insere(9);

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	89



Colisões - Sondagem Linear

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

Insere(89);
Insere(18); *H(18,10)=8*
Insere(49);
Insere(58);
Insere(9);

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	89



Colisões - Sondagem Linear

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

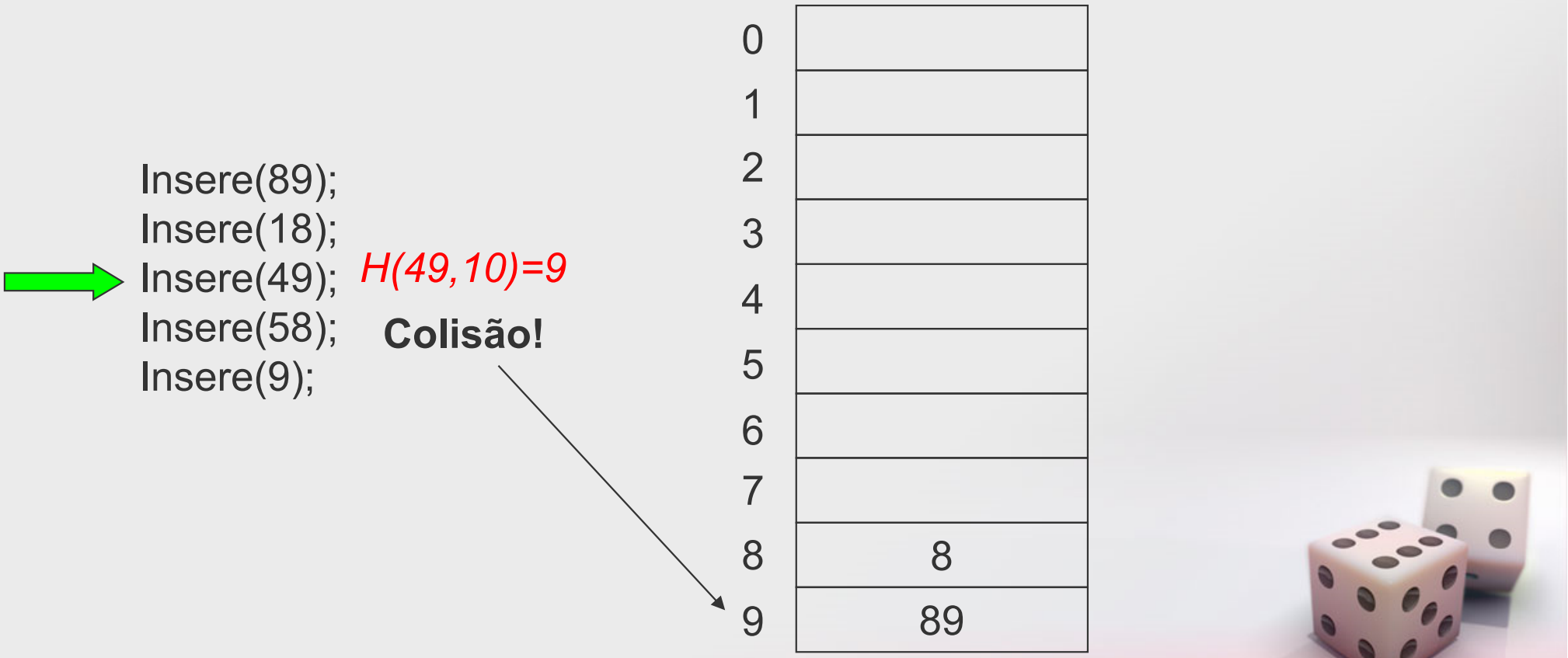
Insere(89);
Insere(18); *H(18,10)=8*
Insere(49);
Insere(58);
Insere(9);

0	
1	
2	
3	
4	
5	
6	
7	
8	8
9	89



Colisões - Sondagem Linear

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.



Colisões - Sondagem Linear

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

Insere(89);
Insere(18);
Insere(49);
Insere(58);
Insere(9);

H(49,10)=9

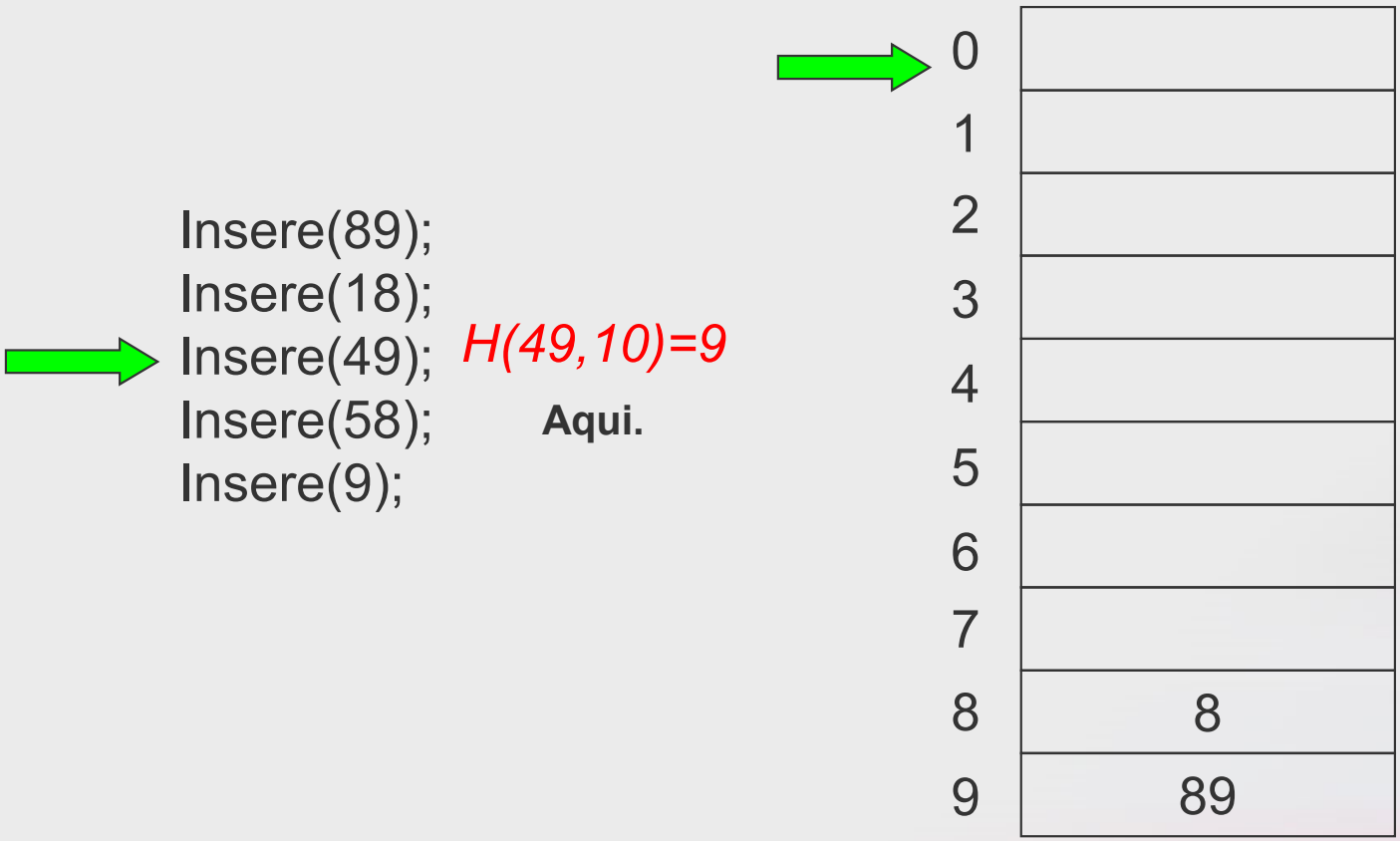
Vamos pesquisar a próxima posição livre!

0	
1	
2	
3	
4	
5	
6	
7	
8	8
9	89



Colisões - Sondagem Linear

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.





Colisões - Sondagem Linear

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

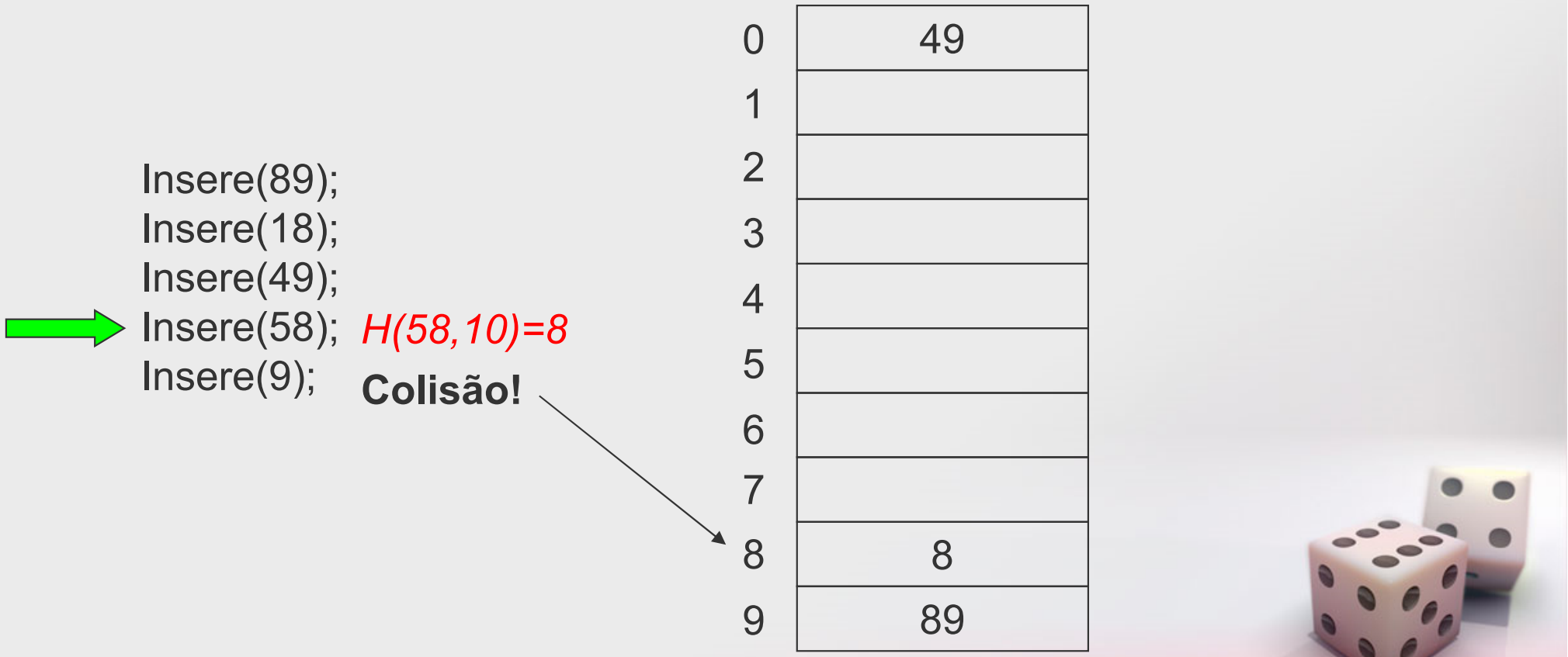
Insere(89);
Insere(18);
Insere(49); *H(49,10)=9*
Insere(58); **Aqui.**
Insere(9);

0	49
1	
2	
3	
4	
5	
6	
7	
8	8
9	89



Colisões - Sondagem Linear

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.



Colisões - Sondagem Linear

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

Insere(89);
Insere(18);
Insere(49);
Insere(58); *H(58,10)=8*
Insere(9); sondagem linear...

0	49
1	
2	
3	
4	
5	
6	
7	
8	8
9	89



Colisões - Sondagem Linear

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

Insere(89);
Insere(18);
Insere(49);
Insere(58); *H(58,10)=8*
Insere(9); sondagem linear...

0	49
1	
2	
3	
4	
5	
6	
7	
8	8
9	89



Colisões - Sondagem Linear

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

Insere(89);
Insere(18);
Insere(49);
Insere(58); *H(58,10)=8*
Insere(9); sondagem linear...

0	49
1	
2	
3	
4	
5	
6	
7	
8	8
9	89



Colisões - Sondagem Linear

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

Insere(89);
Insere(18);
Insere(49);
Insere(58); *H(58,10)=8*
Insere(9); Aqui...

0	49
1	
2	
3	
4	
5	
6	
7	
8	8
9	89



Colisões - Sondagem Linear

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

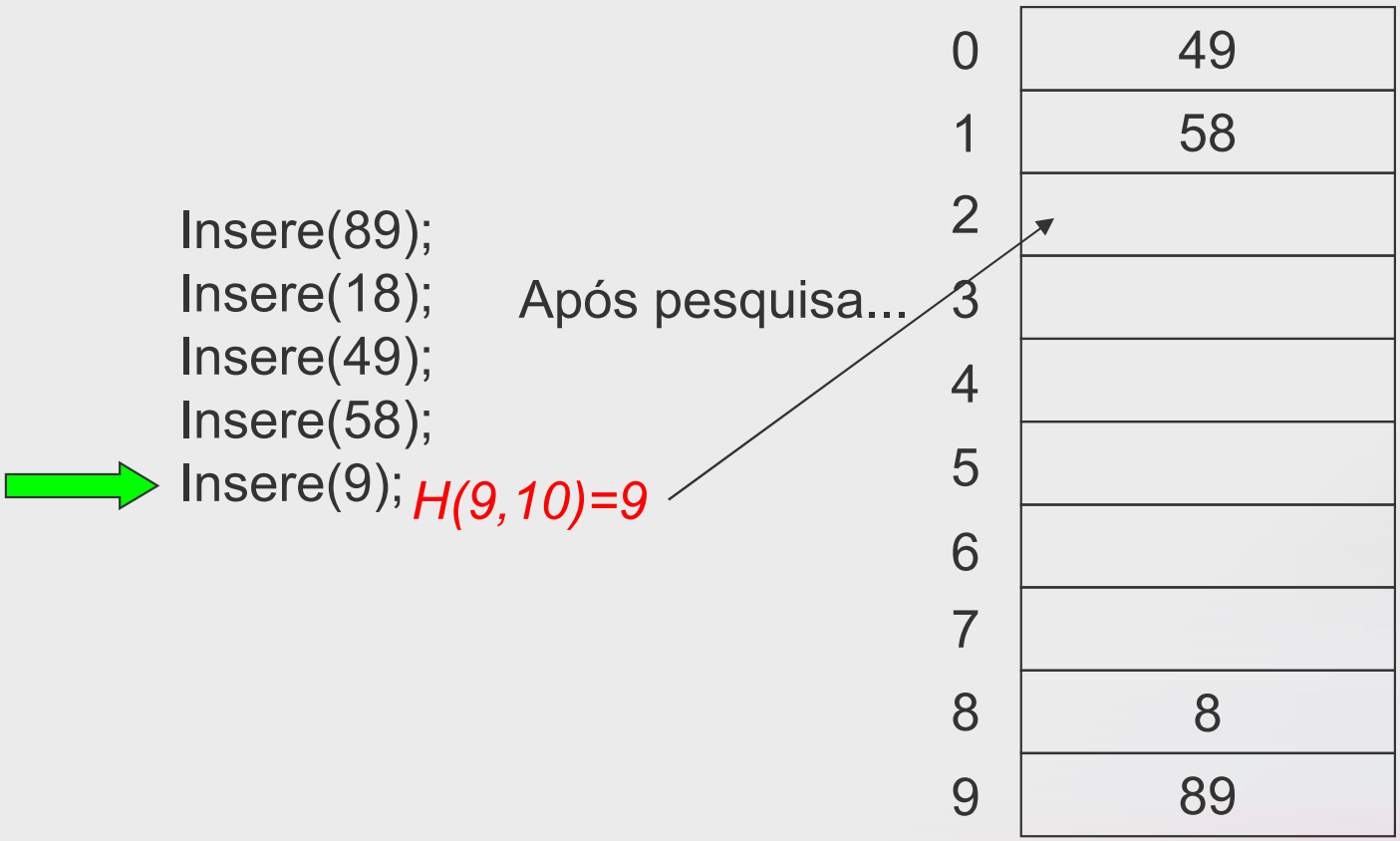
Insere(89);
Insere(18);
Insere(49);
Insere(58); *H(58,10)=8*
Insere(9); Aqui...

0	49
1	58
2	
3	
4	
5	
6	
7	
8	8
9	89



Colisões - Sondagem Linear

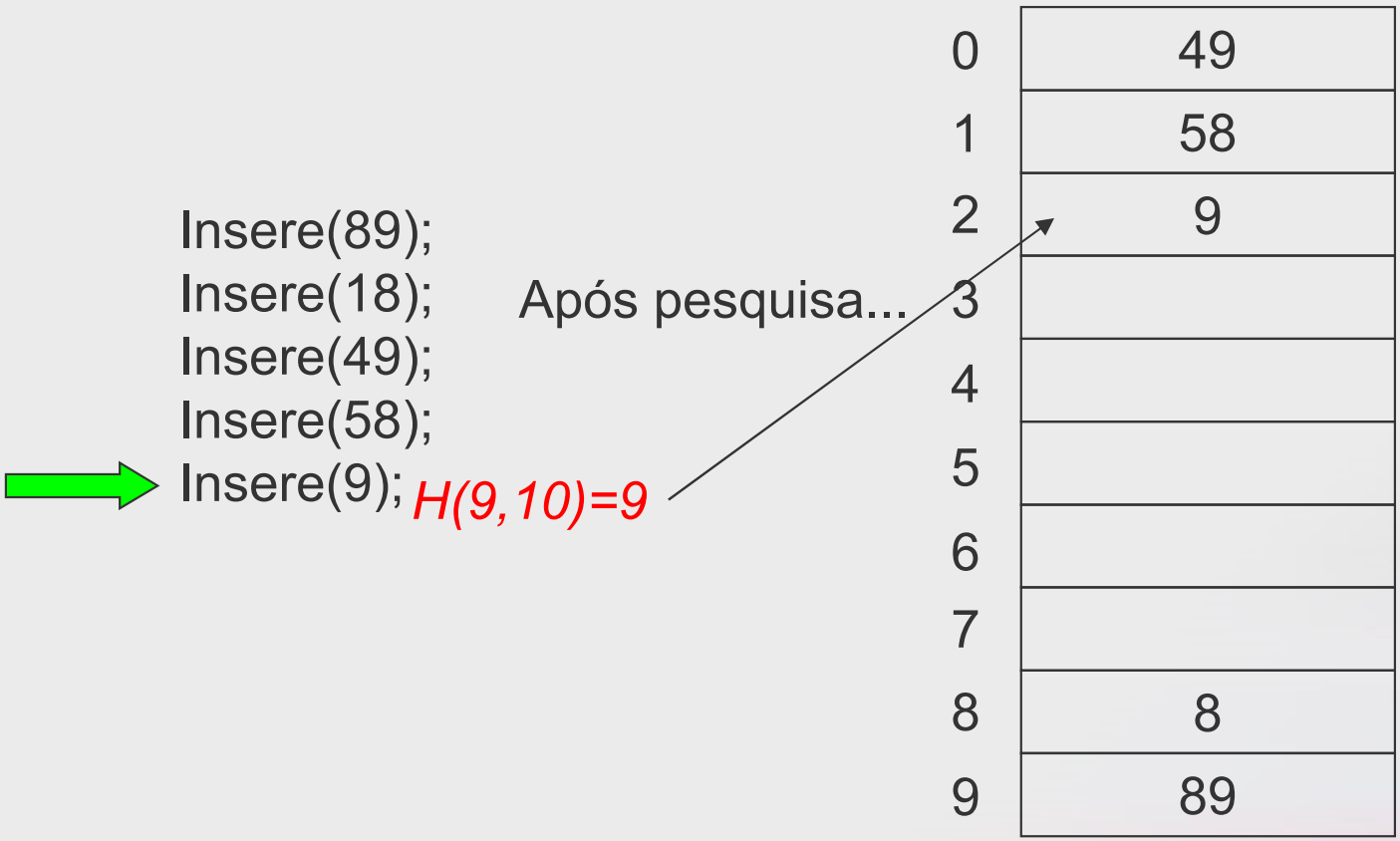
- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.





Colisões - Sondagem Linear

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.





Colisões - Sondagem Linear

- Desde que a tabela não esteja cheia, será sempre encontrada uma célula livre.
- Se a tabela estiver muito cheia, poderá ser necessário percorrer muitos elementos
  - Em média, para uma tabela perto da capacidade máxima, será necessário percorrer cerca de N/2 elementos.
    - O que está bastante longe do tempo constante pretendido.
- No entanto, caso a tabela se mantenha relativamente desocupada, os resultados são bons.



Colisões - Sondagem Linear

- Análise:
  - Caso a ocupação de células seja independente, o número de células percorrido pela sondagem linear é dado por:

$$1/(1-\lambda)$$

onde λ é a taxa de ocupação da tabela.

  - No entanto, a ocupação das células não é independente....
    - Costumam formar-se *clusters* de células.
    - Este valor dá-nos por isso apenas uma indicação do desempenho máximo, em condições perfeitamente ideais.



Colisões - Sondagem Linear

- **Análise:**
  - O efeito de *clustering* faz com na prática o número de células médio analisado seja dado por:

$$\frac{1}{2}\left(1+\frac{1}{(1-\lambda)^2}\right)$$

taxa de ocupação	# células
0	1,0
0,1	1,1
0,2	1,3
0,3	1,5
0,4	1,9
0,5	2,5
0,6	3,6
0,7	6,1
0,8	13,0
0,9	50,5



Colisões - Sondagem Linear

- **Análise (Pesquisa):**
  - O custo de uma pesquisa sem sucesso é idêntico ao custo da inserção mediante a taxa de coupação actual.
  - O custo de uma pesquisa com sucesso é idêntico ao custo da inserção mediante a taxa de ocupação quando esse valor foi inserido. É por isso menor.
    - Sabendo isso, calculando o valor médio, verifica-se que o número de células percorrido em média por uma pesquisa com sucesso é dado por:

$$\frac{1}{2}\left(1+\frac{1}{1-\lambda}\right)$$

taxa de ocupação	# células
0	1,0
0,1	1,1
0,2	1,1
0,3	1,2
0,4	1,3
0,5	1,5
0,6	1,8
0,7	2,2
0,8	3,0
0,9	5,5



Sondagem Quadrática

- A sondagem linear funciona muito bem para taxas de ocupação reduzidas
  - Os problema de desempenho para taxas de coupação elevada são causados por *clustering* dos dados.
- Métodos alternativos propõem melhorar esse aspecto sem aumentar a complexidade do algoritmo.
- Um desses algoritmos é a sondagem quadrática
  - A ideia base é efectuar a pesquisa de forma não sequencial de forma a evitar a agregação de items de dados





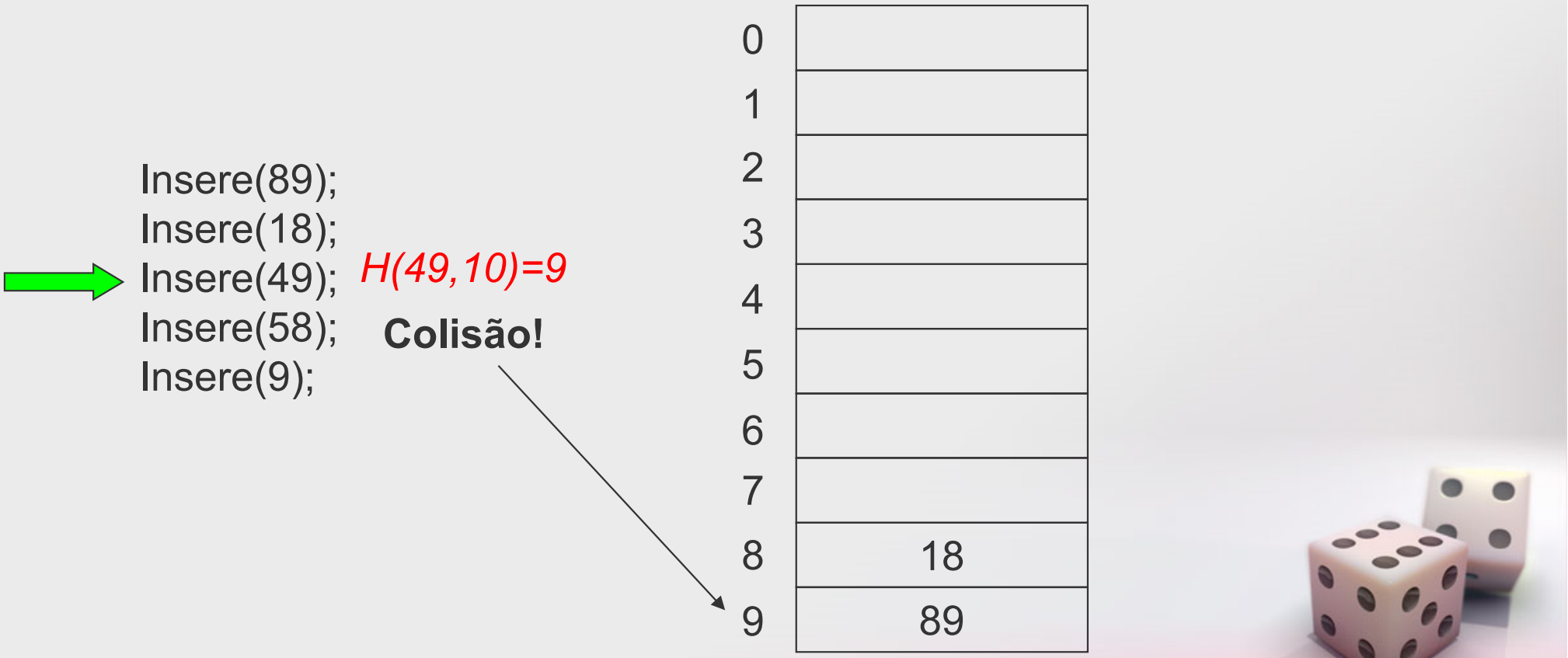
sondagem quadrática

- Caso a posição H esteja ocupada:
  - Pesquisa Sequêncial:
    - *Procura-se nas posições: H+1, H+2, H+3, ...*
  - sondagem quadrática*:*
    - *Procura-se nas posições: $H+1^2$, $H+2^2$, $H+3^2$,...*



sondagem quadrática

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.



sondagem quadrática

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

$(9+1^2)$ mod 10 =0

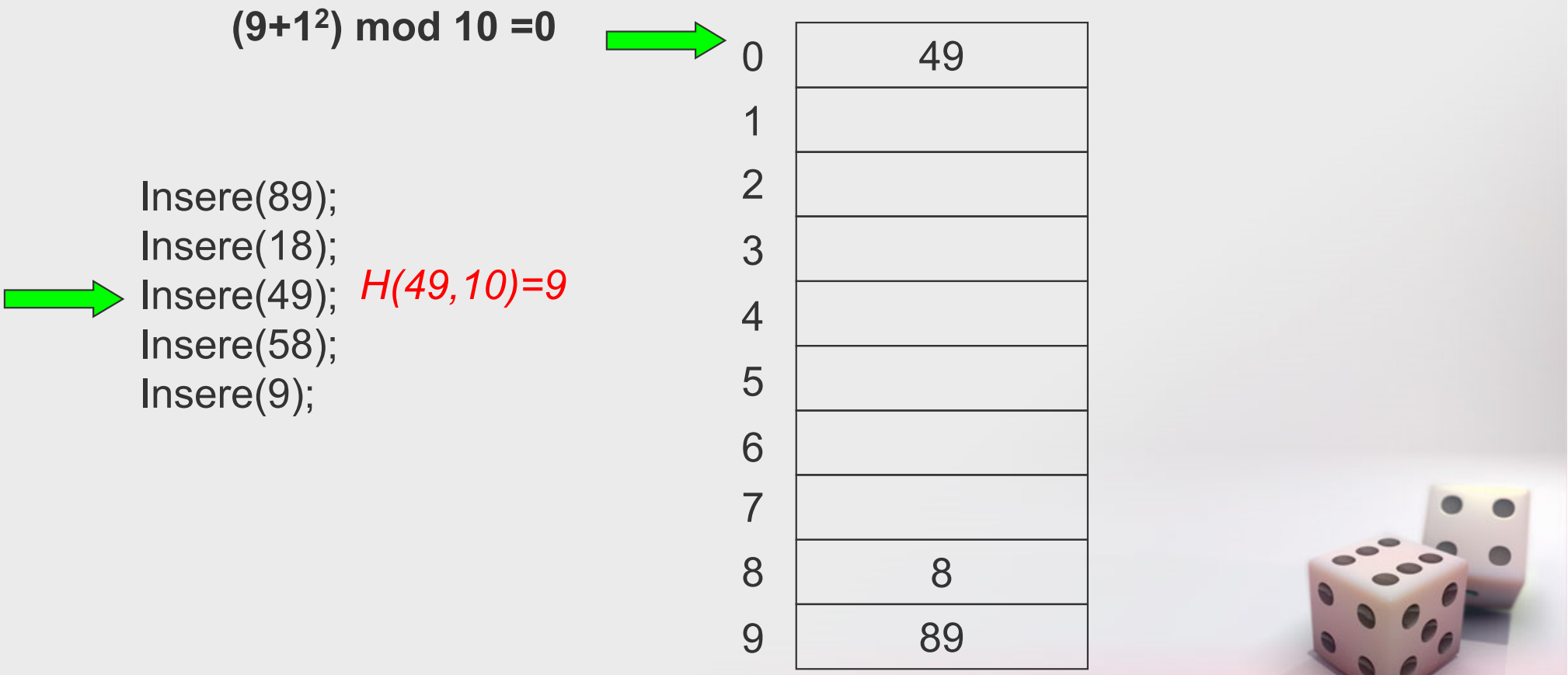
Insere(89);
Insere(18);
Insere(49); *H(49,10)=9*
Insere(58);
Insere(9);

0	
1	
2	
3	
4	
5	
6	
7	
8	18
9	89



sondagem quadrática

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.



sondagem quadrática

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

Insere(89);
Insere(18);
Insere(49);
Insere(58); *H(58,10)=8*
Insere(9);

0	49
1	
2	
3	
4	
5	
6	
7	
8	8
9	89

$(8+1^2)$ mod 10 =9



sondagem quadrática

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

$(8+2^2)$ mod 10 =2

Insere(89);
Insere(18);
Insere(49);
Insere(58); *H(58,10)=8*
Insere(9);

0	49
1	
2	
3	
4	
5	
6	
7	
8	8
9	89



sondagem quadrática

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

$(8+2^2)$ mod 10 =2

Insere(89);
Insere(18);
Insere(49);
Insere(58); *H(58,10)=8*
Insere(9);

0	49
1	
2	58
3	
4	
5	
6	
7	
8	8
9	89



sondagem quadrática

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

$(9+1^2)$ mod 10 =0

Insere(89);
Insere(18);
Insere(49);
Insere(58);
Insere(9); *H(9,10)=9*

0	49
1	
2	58
3	
4	
5	
6	
7	
8	8
9	89



sondagem quadrática

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

Insere(89); **$(9+2^2)$ mod 10 =3**
Insere(18);
Insere(49);
Insere(58);
Insere(9); *H(9,10)=9*

0	49
1	
2	58
3	
4	
5	
6	
7	
8	8
9	89



sondagem quadrática

- Exemplo:
  - Considere-se uma função de *hash H(N,M)=N%M*.

Insere(89); **$(9+2^2)$ mod 10 =3**
Insere(18);
Insere(49);
Insere(58);
Insere(9); *H(9,10)=9*

0	49
1	
2	58
3	9
4	
5	
6	
7	
8	8
9	89



sondagem quadrática

- Será que funciona mesmo?
  - Percorremos todas as células?
    - Se existir apenas uma célula livre, conseguimos encontrá-la?
    - Passamos por cada célula uma única vez?



sondagem quadrática

- Será que funciona mesmo?
  - Percorremos todas as células?
    - Se existir apenas uma célula livre, conseguimos encontrá-la?
    - Passamos por cada célula uma única vez?

Sim, se a dimensão da tabela for prima e a carga menor do que 0.5



sondagem quadrática

- Eliminando os *clusters*, temos um ganho potencial de 0.5 acessos quando a taxa de ocupação é de 0.5
  - 2 (ideal) vs. 2.5(p.linear)
- O ganho é muito reduzido:
  - Dado que a implementação parece necessitar um uma multiplicação, uma soma e o cálculo do resto, será que vale mesmo a pena?...



sondagem quadrática

- Implementação:
  - Felizmente, a implementação da sondagem quadrática é mais simples do que aparenta ser...
  - Se H_{i-1} for a última posição calculada e pretende-se calcular H_i (H_0é a pos. original), então:

$$H_i = H_0 + i^2$$

$$H_{i-1} = H_0 + (i-1)^2$$

Sabendo isso e subtraindo...



sondagem quadrática

- Implementação:

O indice pode ser calculado a partir de:

$$H_i = H_{i-1} + (2i-1) (\text{mod } M)$$

Como 2i-1 é menor do que *M* (porquê?), a operação *mod* pode ser efectuada com um teste e subtracção (como?)



Redimensionamento

- A *hash table* pode ser redimensionada quando antinge uma taxa de ocupação de 0.5:
  - Para melhor o desempenho, no caso da sondagem linear.
    - Tipicamente, o tamanho é aumentado para o dobro
  - Para garantir operação correcta, no caso da sondagem quadrática
    - Neste caso, é necessário encontrar o próximo número primo maior do que o dobro do tamanho actual:
      - Esta operação demora $O(N^{1/2} \log N)$. (menor que O(N))



Redimensionamento

- Será possível simplesmente copiar os items para a nova tabela?



Redimensionamento

- Será possível simplesmente copiar os items para a nova tabela?
  - Não: a função de *hash* depende da dimensão da tabela
  - É necessário recalcular de novo os valores de *hash* e reintroduzir a informação.
  - Esta operação chama-se *rehashing*.



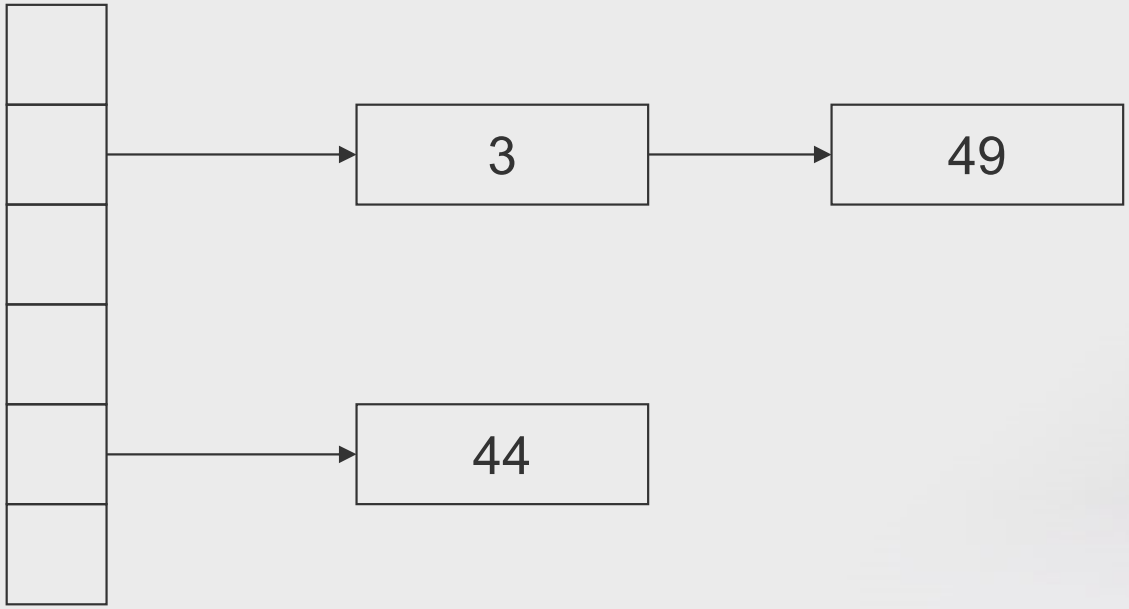
Encadeamento

- As técnicas de sondagem não funcionam bem para taxas de ocupação elevadas.
  - Obrigam por isso ao redimensionamento das tabelas.
- Caso o redimensionamento seja indesejável, é possível usar a técnica de *encadeamento* para lidar com colisões.



Encadeamento

- Neste caso, em cada posição da tabela, há uma lista que contém todos os elementos que partilham esse indice *hash*.





Encadeamento

- No caso de encadeamento, a taxa de ocupação pode ser maior do que 1.
  - O comprimento médio das listas é igual à taxa de ocupação λ.
    - O número médio de células percorridas para inserção é λ
    - O número médio de células percorridas para uma pesquisa com sucesso é 1+ λ/2
  - A taxa de carga típica é 1.0



